Davante

6

PROGRAMACIÓN

Estructuras de almacenamiento y cadenas de caracteres

ÍNDICE

/ 1. Introducción y contextualización práctica 4

/ 2. Datos compuestos 5

2.1. Arrays 5

/ 3. Declaración de arrays 6

3.1. Inicialización de arrays 6

/ 4. Caso práctico 1: “Nota media” 7

/ 5. Referencias 8

5.1. Recolector de basura 9

/ 6. Ordenación de arrays 10

/ 7. Búsqueda de elementos 11

7.1. Búsqueda lineal 11

7.2. Búsqueda binaria 12

/ 8. Tablas bidimensionales 12

8.1. Declaración y creación de arrays multidimensionales 12

8.2. Acceso a elementos 13

8.3. Inicialización 13

8.4. Iteración 13

/ 9. Cadenas de caracteres 14

9.1. Secuencias de escape 14

ÍNDICE

/ 10. Clase Character 15

/ 11. Caso práctico 2: “Contar vocales” 16

/ 12. Clase String 17

12.1. Cálculo de longitud 18

12.2. Más operaciones con cadenas de caracteres 19

/ 13. Resumen y resolución del caso práctico de la unidad 20

/ 14. Bibliografía 21

© MEDAC ISBN: 978-84-19885-69-2
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

OBJETIVOS

Conocer las estructuras de datos.

Comprender el concepto de array.

Saber trabajar con matrices.

Operar con cadenas de caracteres.

/ 1. Introducción y contextualización práctica

En este tema se presentan las estructuras de datos fundamentales para poder agrupar elementos en una única estructura y trabajar con ellos de una forma más sencilla. Gracias a estas estructuras de datos denominadas arrays, es posible crear colecciones de datos en nuestros algoritmos.

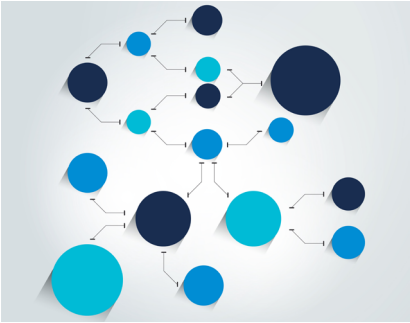


Fig.1. Estructura de datos.

Durante este tema presentamos el concepto de dimensión en las estructuras de datos que permiten ampliar el concepto de matriz. Además, se presentan las cadenas de caracteres, siendo, en ocasiones, consideradas una estructura de colección de datos homogénea o, lo que es lo mismo, un array de caracteres. La utilización de cadenas es uno de los tipos de datos más utilizados a la hora de trabajar con los distintos algoritmos.

A continuación, vamos a plantear un caso práctico a través del cual podremos aproximarnos de forma práctica a la teoría de este tema.

Escucha el siguiente audio donde planteamos la contextualización práctica de este tema. Encontrarás su resolución en el apartado 'Resumen y resolución del caso práctico'.

Audio intro. "Encuentra el valor máximo"

https://on.soundcloud.com/e91e9dZ7z8nN8Duz7

/ 2. Datos compuestos

Los tipos de datos nos permiten definir el dominio de valores que puede tener una determinada variable o constante. En ocasiones, se define un conjunto de valores en el que, por defecto, no existe ningún lenguaje de programación. Además, dependiendo del problema que estamos solucionando, puede ser necesario que el programador disponga de la flexibilidad para poder crear diferentes dominios de valores acordes al problema que está solucionado.

Los **tipos compuestos** pueden definirse teniendo en cuenta los tipos de datos que pueden almacenar, siendo estos capaces de almacenar tanto tipos primitivos como otros tipos de datos compuestos. Algunos ejemplos de tipos de datos compuestos son: lista de valores (arrays), matrices, cadena de caracteres, etc.

2.1. Arrays

Supongamos que en la solución de un problema determinado necesitamos almacenar un conjunto de valores que sean todos del mismo tipo. Además, es necesario que seamos capaces de manejar estos datos utilizando una única variable. Como podemos observar, la idea es que debemos disponer de un contenedor de datos para facilitar la manipulación de estos. Cuando hablamos de un conjunto de datos del mismo tipo, lo definimos como vector, lista o array. Cada uno de los datos almacenados en este vector recibe el nombre de **elementos del vector**.

A la hora de distinguir los distintos elementos que componen nuestro array, cada uno de ellos tiene un **índice** que indica su posición en el array, teniendo el primer elemento asignado el índice 0, el segundo el índice 1, y así sucesivamente hasta llegar al último elemento, que tiene como índice el número de elementos menos uno, como se puede observar en la siguiente imagen:

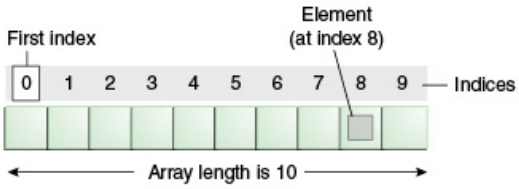


Fig.2. Tabla de 10 elementos.

Una vez vista la imagen previa, surge una pregunta. ¿Qué pasaría si intentamos acceder a un índice que se encuentra fuera del rango 0 a 9? En ese caso, saltaría un **error en tiempo de compilación**.

Las **operaciones más usuales** que vamos a realizar con arrays son las siguientes:

- Creación e inicialización.
- Agregar elementos.
- Eliminar elementos.
- Realizar búsquedas de elementos.
- Ordenar el array.

/ 3. Declaración de arrays

Al igual que sucede con otros tipos de datos, para poder utilizar variables o constantes de tipo array, es necesario poder declarar dichas variables anteriormente, indicando el tipo de datos que va a almacenar el array. Para poder declarar una variable de tipo array, es preciso indicar al menos los siguientes parámetros:

- **Tipo de datos**: Hace referencia al tipo de los datos de cada uno de los elementos de la estructura. Recordemos que todos los elementos deben ser del mismo tipo.
- **Longitud**: Permite indicar el número de elementos que vamos a poder almacenar en nuestra lista.
- **Nombre variable**: Es el identificador que hace referencia a la variable. Como todas las variables y constantes, a la hora de declarar una lista, tenemos que indicar el nombre de esa lista para poder referenciarlos a ellas en el futuro a lo largo de nuestro código.

El primer paso para la creación del array es su declaración utilizando los corchetes []. Se pueden emplear dos tipos de sintaxis:

```
tipo nombreVariable[];
tipo[] nombreVariable;
```

Un ejemplo de declaración para un array de enteros sería:

```
int edades[];
int[] edades;
```

Una vez declarada la variable, se creará el array con la longitud que necesitamos, asignándoselo a la variable creada:

```
nombreVariable = new tipo[longitud];
edades = new int[9];
```

¡! Sabías que...

Mediante el uso del operador new se crea un array con todos los elementos a 0 en el caso de ser de tipo numérico y false en caso de ser de tipo boolean.

3.1. Inicialización de arrays

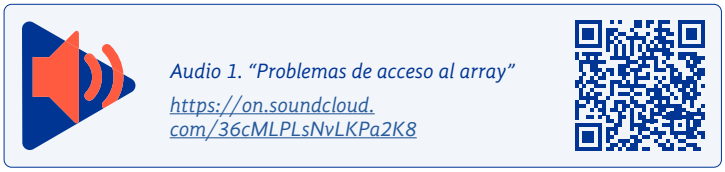
Existen tres formas de inicializar un array:

- **Por posición**: Una forma bastante rudimentaria de poder inicializar una lista es haciendo referencia a cada uno de los elementos que la componen. De esta forma, podríamos asignar valor a cada una de las posiciones, haciendo referencia a cada una de ellas.

- **Por iteración**: Se podría asignar un valor a cada uno de los elementos de una lista utilizando un bucle de iteración.
- **Existe una manera de crear arrays sin necesidad de emplear el operador *new***: Consiste en darle los valores de inicio, por ejemplo:

```
int edades[] = {11,12,13,14,15,16,17,18,19};
```

Mediante esta sentencia se declara la variable edades y se crea una tabla de nueve enteros.



/ 4. Caso práctico 1: "Nota media"

Planteamiento: Los arrays permiten almacenar múltiples valores relacionados bajo un único nombre de variable, lo que facilita la gestión y organización de los datos, ya que se puede acceder y manipular fácilmente usando índices, permitiendo realizar operaciones como calcular la suma o la media de manera sencilla y eficiente.

Nudo: Se pide escribir un programa que:

- Solicite al usuario el tamaño del array.
- Solicite al usuario los valores para llenar el array.
- Calcule la media de todos los elementos del array.
- Imprima la media calculada.

Desenlace:

```java
import java.util.Scanner;

public class MediaArray {
    public static void main(String[] args) {
        // Crear un objeto Scanner para leer la entrada del usuario
        Scanner scanner = new Scanner(System.in);

        // Solicitar al usuario el tamaño del array
        System.out.print("Introduce el tamaño del array: ");
        int tamaño = scanner.nextInt();

        // Crear un array de enteros con el tamaño especificado
        int[] numeros = new int[tamaño];

        // Leer los valores del usuario y llenar el array
        System.out.println("Introduce " + tamaño + " valores:");
        for (int i = 0; i < tamaño; i++) {
            System.out.print("Valor " + (i + 1) + ": ");
            numeros[i] = scanner.nextInt();
        }

        // Calcular la suma de todos los elementos del array
        int suma = 0;
        for (int i = 0; i < tamaño; i++) {
            suma += numeros[i];
        }

        // Calcular la media
        double media = (double) suma / tamaño;

        // Imprimir la media
        System.out.println("La media de los valores introducidos es: " + media);
    }
}
```

Fig.3. Resolución caso práctico 1.

Una posible solución sería la mostrada en la imagen 3.

/ 5. Referencias

Hasta ahora hemos creado un array, pero no sabemos cómo funciona de manera interna. A continuación se va a crear un array y analizar qué sucede al emplear el operador new:

```
int datos[] = new int[5];
```

De manera general se podría decir que mediante esta sentencia se ha creado un array de tipo entero con cinco elementos y se le ha asignado a la variable datos, sin embargo, hay que tener en cuenta los siguientes **pasos al emplear el operador new**:

1. Se calcula el tamaño necesario para almacenar cinco elementos de tipo entero, por tanto, y teniendo en cuenta que ocupan posiciones contiguas en memoria, para calcular la memoria necesaria, se deberá multiplicar el tamaño de un tipo entero por el número de valores que almacena. Si el tamaño de un entero es 4 bytes y estamos almacenando 5, se deberán reservar 20 bytes en memoria.

2. Una vez conocido el espacio requerido, se debe buscar en memoria un hueco de al menos 20 bytes para almacenar el array.

3. Una vez localizado el espacio en memoria, se almacena y se marca esa memoria como ocupada.

4. Por último, se recorren los elementos y se inicializan.

La siguiente figura 4 muestra la representación de la memoria antes y después de crear el array de 5 elementos, teniendo en cuenta que la memoria ocupada está en azul, la zona libre en blanco, y la zona en verde la destinada al array creado.

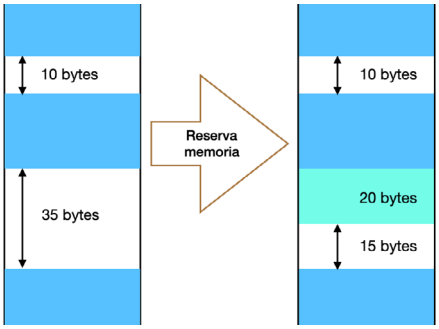


Fig.4. Representación en memoria.

Una vez creada la tabla, se debe indicar qué posición ocupa en memoria. Al tener posiciones contiguas en memoria, tan solo es necesario indicar la primera posición del array. A esta dirección se le denomina referencia y funcionaría de una manera similar a la que muestra la figura 5.

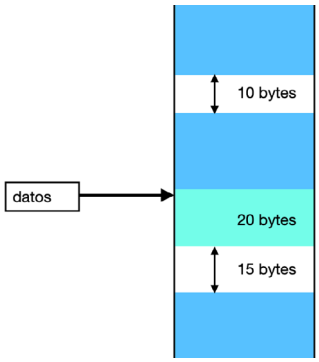


Fig.5. Referencia a la primera posición en memoria.

5.1. Recolector de basura

Ahora que sabemos cómo funcionan las matrices y las referencias, vamos a plantear el siguiente caso:

```
int datos[], productos[], precios[];
datos[] = new int[3];
productos [] = new int[4];
new int[5];
```

En el **ejemplo**, se reservan tres espacios en memoria correspondientes a los tres new que se ejecutan, sin embargo, **el último, a pesar de construirse, no se está asignando a ninguna variable** y, por tanto, no hay manera de acceder a esa tabla, obteniendo la representación que se muestra en la siguiente imagen (figura 6).

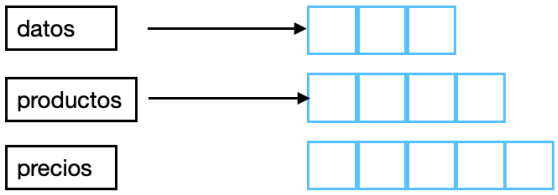


Fig.6. Referencias a tablas.

Otra opción que puede ocurrir es que tengamos más de una manera de acceder a una tabla, es decir, **tener más de una referencia a una tabla**. Observa el siguiente código y su imagen (figura 7).

```
precios = datos;
```

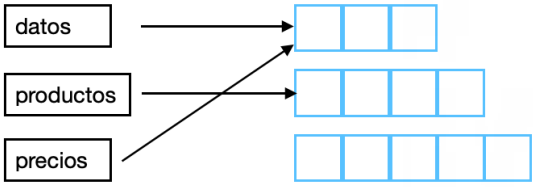


Fig.7. Referencia múltiple.

O, por el contrario, que una variable con una referencia asignada a una tabla deje de hacerlo mediante el **uso de la palabra reservada null** (figura 8)

```
datos = null;
```

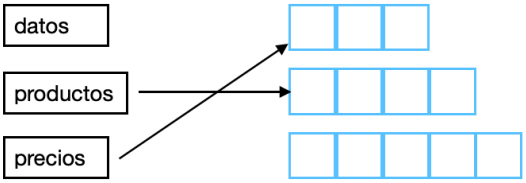


Fig.8. Referencia a null.

En el caso de que exista un array que no tenga referencia y, por tanto, no puede ser accedido o deje de ser utilizado, Java se encarga de liberar esa memoria ocupada mediante el uso del 'garbage collector'. Es decir, **el garbage collector se encargará de liberar la memoria de los arrays que no se están utilizando**.

/ 6. Ordenación de arrays

Una de las operaciones más habituales que se realizan con arrays es la ordenación. La forma en que podemos ordenar los elementos de un array puede ser la que necesitemos, pero lo más normal es ordenarlos en orden ascendente o descendente.

Existen tres **métodos básicos de ordenación** de arrays:

- **Método de la burbuja**: Compara pares de elementos adyacentes y los intercambia si están en el orden incorrecto. Este proceso se repite hasta que no se realizan más intercambios, lo que indica que el array está ordenado.

- **Método de selección**: Busca repetidamente el elemento más pequeño (o más grande) de la porción no ordenada del array y lo coloca al principio (o al final) de la porción ordenada.

- **Método de inserción**: Construye una secuencia ordenada de elementos uno a uno, insertando cada nuevo elemento en su lugar adecuado dentro de la secuencia ya ordenada.

Estos tres métodos son los más básicos, es decir, son los más fáciles de implementar, pero son los menos eficientes en cuanto a tiempo se refiere.

Si utilizamos estos métodos con arrays de pocos elementos, no será significativo el tiempo que se emplea cuando operamos con ellos, pero si los utilizamos con arrays con una cantidad considerable de elementos, podremos ver que tardan muchísimo en realizar las operaciones.

La eficiencia en orden de tiempo de un algoritmo se mide mediante la **notación de complejidad O** (O mayúscula). La de estos tres métodos sería $O(n^2)$, siendo n la cantidad de elementos que tenga el array. Por ejemplo, para un array de 100 elementos, la ordenación en cualquiera de estos métodos entrañaría una complejidad en torno a 100^2 = 10.000.

Existen métodos más eficientes y menos complejos para la ordenación, como son Mergesort o Quicksort, entre otros.

¡! Sabías que...

Quicksort es un algoritmo de ordenamiento rápido y eficiente que utiliza la técnica de dividir y vencerás. Mergesort también usa dividir y vencerás, pero garantiza estabilidad y complejidad O(n log n).

A continuación se muestra el código empleado para medir el tiempo de ejecución (figura 9).

```
long startTime = System.currentTimeMillis();

// Código a medir.
// ...
//
long endTime = System.currentTimeMillis() - startTime;
System.out.println("Tiempo de ejecución total: " + endTime + "ms.");
```

Fig.9. Medir el tiempo de ejecución.

Vídeo 1. "Ordenación de un array"
https://bit.ly/335cqP8

/ 7. Búsqueda de elementos

Las búsquedas de elementos en los arrays son algo fundamental. Mediante las búsquedas, podremos comprobar si un elemento existe dentro de un array o no.

7.1. Búsqueda lineal

El primer tipo de búsqueda que vamos a ver es la búsqueda lineal. Esta búsqueda es la más fácil de entender y de programar, pero, al igual que pasaba con los métodos de ordenación, al ser la más simple de implementar, será la menos eficiente (O (n)), es decir, **la que más tarda en encontrar el elemento**.

El pseudocódigo de la búsqueda es el siguiente:

- Recorremos el array.
- Si el elemento por donde vamos es igual al que buscamos, paramos.
- Aumentamos el contador de la posición y volvemos al punto 1.

La siguiente imagen (figura 10) muestra el código resultante:

```java
/**
 * Método que aplica la búsqueda lineal a un array
 *
 * @param array Array para buscar un elemento
 * @param elemento Elemento que se busca en el array
 * @return Devuelve true si el elemento está en el array, falso sino
 */
public static boolean busquedaLineal(int array[], int elemento) {
    boolean encontrado = false;

    for (int i = 0; i < array.length && !encontrado; i++) {
        if (array[i] == elemento) {
            encontrado = true;
        }
    }

    return encontrado;
}

public static void main(String[] args) {
    int[] array = new int[10]; // Declaración de un array
    // Relleno el array
    for (int i = 0; i < array.length; i++) {
        array[i] = i + 1;
    }

    // Busco el 4 en el array
    boolean estael4 = busquedaLineal(array, 4);
}
```

Fig.10. Búsqueda lineal.

7.2. Búsqueda binaria

La búsqueda binaria es un tipo de búsqueda en arrays **mucho más eficiente** (O (log n)) que la búsqueda lineal, pero algo más compleja de implementar, y además **necesita que el array esté obligatoriamente ordenado**.

Se basa en tomar los extremos del array y comparar el elemento central con el que estamos buscando. Si son iguales, la búsqueda termina con éxito. En caso contrario, se comprueba si el elemento buscado es menor o mayor que este. Como el array está ordenado, si es menor, el elemento que buscamos estará en el lado izquierdo del array; si es mayor, en el derecho. En ambos casos se continúa la búsqueda en el lado que corresponda, actualizando los extremos y volviendo a comparar el elemento central del nuevo segmento. El proceso se repite hasta encontrar el elemento o hasta que los extremos del segmento se crucen, que implicará que el elemento no se encuentre en el array.

La siguiente imagen (figura 11) muestra el código resultante:

```java
/**
 * Método que aplica la búsqueda binaria a un array
 *
 * @param array Array para buscar un elemento
 * @param elemento Elemento que se busca en el array
 * @return Devuelve true si el elemento está en el array, falso sino
 */
public static boolean busquedaBinaria(int array[], int elemento) {
    boolean encontrado = false;
    int inicio = 0;
    int fin = array.length - 1;
    int posicion_buscar;

    while (inicio <= fin && !encontrado) {
        posicion_buscar = (inicio + fin) / 2;
        if (array[posicion_buscar] == elemento) {
            encontrado = true;
        } else {
            if (elemento > array[posicion_buscar]) {
                inicio = posicion_buscar + 1;
            } else {
                fin = posicion_buscar - 1;
            }
        }
    }

    return encontrado;
}
```

Fig.11. Búsqueda binaria.

/ 8. Tablas bidimensionales

Son **arrays que contienen otros arrays como elementos**, lo que permite organizar los datos de manera estructurada en múltiples dimensiones. Son considerados como matrices o tablas con filas y columnas, aunque pueden tener más de dos dimensiones.

8.1. Declaración y creación de arrays multidimensionales

La sintaxis para declarar y crear un array multidimensional en Java es similar a la de un array unidimensional, pero con corchetes adicionales para cada dimensión.

```java
int[][] matriz = new int[3][4];
```

Este código crea una matriz de 3 filas y 4 columnas. Cada elemento de esta matriz es un array unidimensional de enteros.

8.2. Acceso a elementos

Podemos acceder a los elementos de un array multidimensional utilizando múltiples índices correspondientes a cada dimensión.

```
matriz[0][0] = 1; // Establecer el valor del
elemento en la primera fila y primera columna

int valor = matriz[1][2]; // Obtener el valor del
elemento en la segunda fila y tercera columna
```

8.3. Inicialización

También podemos inicializar un array multidimensional directamente.

```
int[][] matriz = {
   {1, 2, 3},
   {4, 5, 6},
   {7, 8, 9}
};
```

8.4. Iteración

Para recorrer un array multidimensional, podemos emplear bucles anidados.

```
for (int i = 0; i < matriz.length; i++) {
   for (int j = 0; j < matriz[i].length; j++) {
      System.out.print(matriz[i][j] + " ");
   }
   System.out.println(); // Nueva línea para cada
fila
}
```

Algortimo 1. Recorrer array bidimensional.

Java también permite crear arrays **multidimensionales** con más de dos dimensiones. Por ejemplo, un array tridimensional se puede declarar y crear de la siguiente manera:

```
int[][][] cubo = new int[3][3][3];
```

Los arrays multidimensionales son útiles para representar datos en múltiples dimensiones, como matrices, cubos, imágenes, etc. Su uso puede simplificar la gestión de datos estructurados y permitir operaciones más eficientes en ciertos casos.

¡! Sabías que...

Un caso práctico de tablas multidimensionales en la vida real es la gestión de datos en un sistema de reservas de vuelos. Donde: primera dimensión: diferentes vuelos; segunda dimensión: filas de asientos en el avión, y tercera dimensión: columnas de asientos en cada fila.

/ 9. Cadenas de caracteres

Hasta ahora hemos empleado los tipos primitivos, siendo el tipo char el encargado de almacenar un carácter, algo innecesario si es necesario almacenar textos. En este caso se emplea un nuevo tipo denominado cadena de caracteres. Las **cadenas de caracteres** consisten en una **secuencia de caracteres que pertenecen a un determinado alfabeto de símbolos**.

Podemos decir que una cadena de caracteres es una especialización de un array de datos en la que el tipo de los elementos que la forman son caracteres. Al tratarse de una lista de datos, podríamos realizar todas las operaciones que se pueden hacer con las listas.

Podemos crear un **array de tipo char** de la siguiente forma:

```
char[] cadena = new char[10];
```

Con esta expresión, se ha declarado una cadena de caracteres de tamaño 10.

En Java se emplea el estándar **Unicode** para representar caracteres de texto. Esta codificación de caracteres asigna un número único (llamado punto de código) a cada carácter, independientemente del idioma o la plataforma, permitiendo representar una amplia gama de caracteres, incluidos los alfabetos, símbolos, emojis y otros caracteres especiales.

9.1. Secuencias de escape

Un tipo de caracteres especial es la barra invertida conocida como secuencia de escape. Las secuencias de escape son **combinaciones de caracteres que se utilizan para representar caracteres especiales o codificados Unicode en literales de cadena y caracteres**.

La siguiente tabla 1 muestra las principales secuencias de escape:

Carácter	Descripción
\n	Nueva línea
\t	Tabulador
\’	Comilla simple
\”	Comilla doble
\\	Barra invertida

Tabla 1. Caracteres especiales en Java.

En el siguiente código se muestra un ejemplo de uso:

```
//Salto de linea
char mensaje = '\n';
//Tabulador
char tabla = '\t';
//Comilla simple
char comilla = '\'';
//Comilla doble
char cita = '\"'
//Barra invertida
char directorio =  '\\'
```

/ 10. Clase Character

La clase Character en Java es una **clase envolvente (*wrapper*)** para el tipo primitivo char. Proporciona métodos estáticos para realizar operaciones comunes con caracteres, así como algunos métodos de utilidad.

Existen distintos **tipos** de caracteres:

- **Dígitos**: Constituidos por las diez cifras decimales: 0, 1, 2, 3, 4, 5, 6, 7, 8 y 9.
- **Letras**: Son las letras mayúsculas y minúsculas del abecedario inglés: A, B... Y, Z, a, b... y, z.
- **Caracteres de control**: Representan órdenes de control, como el carácter de fin de línea.
- **Caracteres especiales**: Son los símbolos no incluidos en los grupos anteriores, como los siguientes: @, [,], {,},), (, Ñ, ñ, #, $, %, &, /, ;, :.
- **Caracteres gráficos**: Símbolos con los que se pueden representar figuras elementales.

Algunas de las **operaciones** más comunes con la clase Character:

- **boolean isDigit(char c)**: Indica si el carácter es un dígito, devolviendo true en caso afirmativo o false en caso contrario.
- **boolean isLetter(char c)**: Indica si el carácter es una letra, ya sea mayúscula o minúscula, devolviendo true en caso afirmativo o false en caso contrario.
- **boolean isLetterOrDigit(char c)**: Indica si el carácter es una letra o un dígito, es decir, un carácter alfanumérico.

- **boolean isLowerCase(char c)**: Indica si el carácter es una letra minúscula, devolviendo true en caso afirmativo o false en caso contrario.
- **boolean isUpperCase(char c)**: Indica si el carácter es una letra mayúscula, devolviendo true en caso afirmativo o false en caso contrario.
- **boolean isSpaceChar(char c)**: Indica si el carácter es un espacio.

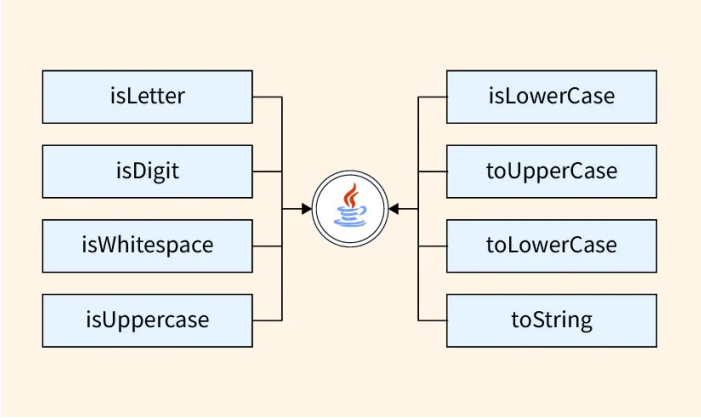


Fig.12. Métodos de la clase Character en Java.

También existen métodos encargados de convertir un carácter en otro:

- **char toLowerCase(char c)**: En caso de que el carácter sea una letra, lo convierte en letra minúscula.
- **char toUpperCase(char c)**: En caso de que el carácter sea una letra, lo convierte en letra mayúscula.

/ 11. Caso práctico 2: "Contar vocales"

Planteamiento: La clase Character y el tipo primitivo char son fundamentales para trabajar con caracteres y textos. Sin embargo, el tipo char es más eficiente en términos de memoria y rendimiento (en comparación con String), y por ello los arrays de tipo char se utilizan comúnmente para manipular cadenas de texto antes de convertirlas a objetos String.

Nudo: Se pide escribir un programa que:

- Solicite al usuario el número de caracteres que va a introducir.
- Permita al usuario ingresar los caracteres uno por uno.
- Almacene estos caracteres en un array de tipo char.
- Cuente cuántas vocales (a, e, i, o, u) hay en el array.
- Imprima el número de vocales y los caracteres del array.

- **Desenlace**: Una posible solución sería la mostrada en la figura 13.

```java
import java.util.Scanner;

public class ContarVocales {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Solicitar al usuario el número de caracteres
        System.out.print("Introduce el número de caracteres: ");
        int numCaracteres = scanner.nextInt();

        // Crear un array para almacenar los caracteres
        char[] caracteres = new char[numCaracteres];

        // Leer los caracteres del usuario
        System.out.println("Introduce los caracteres uno por uno:");
        for (int i = 0; i < numCaracteres; i++) {
            System.out.print("Caracter " + (i + 1) + ": ");
            caracteres[i] = scanner.next().charAt(0);
        }

        // Contar las vocales
        int numVocales = 0;
        for (int i = 0; i< numCaracteres; i++) {
            if (esVocal(caracteres[i])) {
                numVocales++;
            }
        }

        // Imprimir el resultado
        System.out.println("Número de vocales: " + numVocales);
        System.out.println("Caracteres introducidos: ");
        for (char c : caracteres) {
            System.out.print(c + " ");
        }

    }

    // Método para comprobar si un carácter es una vocal
    public static boolean esVocal(char c) {
        c = Character.toLowerCase(c); // Convertir el carácter a minúscula
        return c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u';
    }
}
```

Fig.13. Resolución caso práctico 2.

/ 12. Clase String

La clase String en Java representa una **secuencia inmutable de caracteres**. Esto significa que una vez que se crea una instancia de String, su contenido no puede ser modificado. La clase String proporciona numerosos métodos para trabajar con cadenas de caracteres, incluyendo operaciones de búsqueda, manipulación, comparación y conversión.

La manera de iniciar una cadena de caracteres es análoga a la creación de un objeto y, por tanto, podemos emplear la palabra reservada new.

```
String cadena = new String("Hola mundo");
```

Sin embargo, es tan habitual el uso de la clase String que Java permite abreviar.

```
String cadena = "Hola mundo";
```

Es necesario recordar que los caracteres especiales, como son las comillas, deben ir con una secuencia de escape.

```
String cadena = “Mi perro se llama \”Neron\” ”;
System.out.println(cadena);
```

Alguna de las operaciones que se nos permita realizar con las cadenas de caracteres de una forma especial son:

- Cálculo de longitud.
- Concatenación.
- Subcadena.
- Localización de subcadenas

12.1. Cálculo de longitud

Permite **identificar el número de elementos que forman una cadena de caracteres**. Aunque *a priori* puede parecer una operación relativamente sencilla, es necesario poder establecer un carácter de control para determinar el final de la cadena.

```
// Crear una cadena de caracteres
String miCadena = "Hello";
// Obtener la longitud de la cadena
int longitud = miCadena.length(); // longitud = 5
```

La figura 14 muestra el resultado del código.

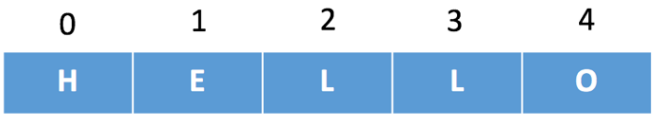


Fig.14. Longitud de la cadena.

Es importante **definir la longitud que puede tener la cadena nula**, es decir, aquella cadena que no tiene ningún valor. Es importante no confundir la cadena nula con una cadena de longitud cero. Aunque conceptualmente son diferentes, la cadena nula también tiene longitud cero.

12.2. Más operaciones con cadenas de caracteres

En la tabla 2, podemos ver algunos de los métodos más útiles que nos ofrece esta clase:

Método	Descripción
length()	Devuelve la longitud de la cadena
indexOf('caracter')	Devuelve la posición de la primera aparición de carácter
lastIndexOf('caracter')	Devuelve la posición de la última aparición de carácter
charAt(n)	Devuelve el carácter que está en la posición n
substring(n1,n2)	Devuelve la subcadena comprendida entre las posiciones n1 y n2-1
toUpperCase()	Devuelve la cadena convertida a mayúsculas
toLowerCase()	Devuelve la cadena convertida a minúsculas
equals("cad")	Compara dos cadenas y devuelve true si son iguales
equalsIgnoreCase("cad")	Igual que equals, pero sin considerar mayúsculas y minúsculas
compareTo(OtroString)	Devuelve 0 si las dos cadenas son iguales. <0 si la primera es alfabéticamente menor que la segunda o >0 si la primera es alfabéticamente mayor que la segunda
compareToIgnoreCase(OtroString)	Igual que compareTo, pero sin considerar mayúsculas y minúsculas
valueOf(N)	Convierte el valor N a String. N puede ser de cualquier tipo
replace (char viejoChar, char nuevoChar)	Remplaza en el string que invoca el método, el viejoChar por el nuevoChar, todas las apariciones
replaceAll(String viejaString, String nuevaString)	Remplaza en el string que invoca al método la viejaString por la nuevaString. Se utiliza para reemplazar subcadenas

Tabla 2. Métodos de la clase String.

A continuación se muestra un ejemplo de uso de estos métodos:

```
// Crear una cadena de caracteres
String miCadena = "Hola, mundo!";

// Obtener la longitud de la cadena
int longitud = miCadena.length(); // longitud == 12

// Obtener un carácter en una posición específica
char primerCaracter = miCadena.charAt(0); // primerCaracter == 'H'

// Concatenar cadenas de caracteres
String nuevaCadena = miCadena + " Qué tal?"; // nuevaCadena == "Hola, mundo! Qué tal?"

// Comparar cadenas de caracteres
boolean sonIguales = miCadena.equals("Hola, mundo!"); // sonIguales == true

// Buscar una subcadena dentro de una cadena
int indice = miCadena.indexOf("mundo"); // indice == 6

// Convertir a mayúsculas
String mayusculas = miCadena.toUpperCase(); // mayusculas == "HOLA, MUNDO!"
```

Fig.15. Operaciones con cadenas.

Vídeo 2. "Expresiones regulares"
https://bit.ly/3jL4jgE

/ 13. Resumen y resolución del caso práctico de la unidad

En este tema se han presentado las estructuras con las que agrupar elementos del mismo tipo en una estructura más compleja de datos. Como hemos visto, las listas o los vectores nos permiten almacenar grandes números de registros. Además, se ha extendido el concepto de array, presentando el término de dimensión con el que aumentar el número de elementos que se pueden almacenar en una estructura homogénea utilizando matrices.

También se ha visto cómo trabajar con los caracteres y las cadenas de datos que ofrecen diferentes tipos de operaciones para llevar a cabo diferentes tareas con facilidad. Entre las operaciones a destacar resalta la concatenación, que es capaz de unir dos cadenas en una única cadena resultante o la longitud que nos muestre la longitud de la cadena de texto, ya que, al fin y al cabo, una cadena de texto no es más que un conjunto de caracteres agrupados, o lo que es lo mismo, un array de tipo carácter.

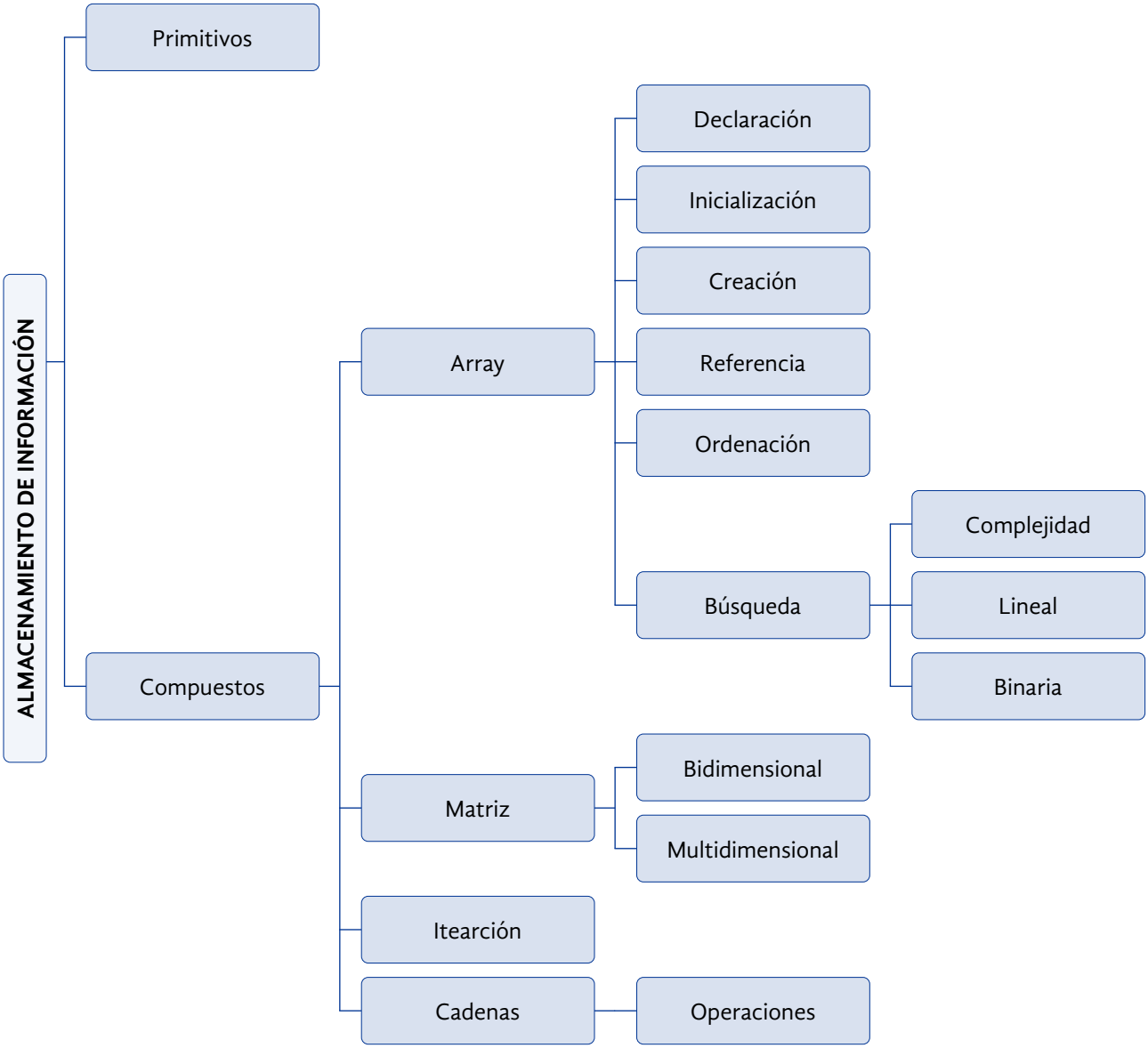


Fig.16. Esquema resumen del tema.

Resolución del caso práctico de la unidad

La figura 17 muestra una implementación de función para buscar el valor máximo de un array.

```
public class MaximoArray {
    public static void main(String[] args) {
        int[] numeros = {5, 8, 3, 12, 7};
        int maximo = encontrarMaximo(numeros);
        System.out.println("El valor máximo del array es: " + maximo);
    }

    // Función para encontrar el valor máximo en un array de enteros
    public static int encontrarMaximo(int[] array) {

        int maximo = array[0]; // Suponemos que el primer elemento es el máximo

        // Iteramos sobre los elementos del array
        for (int i = 1; i < array.length; i++) {
            // Si encontramos un valor mayor, lo actualizamos
            if (array[i] > maximo) {
                maximo = array[i];
            }
        }

        return maximo;
    }
}
```

Fig.17. Valor máximo de un array.

/ 14. Bibliografía

Caro, C. M.; Ramos, A. N., & Barceló, A. V. (2002). *Introducción a la programación con orientación a objetos*. PRENTICE HALL.

Haro, C. (2021) *Algoritmia: Razonar para crear*. Ediciones Eni.

Manuel, P. G. J. (2022). *Entornos de desarrollo*. Ediciones Paraninfo, S. A.

Ojeda, F. C. (2001). *Introducción a la programación*. Anaya Multimedia.

Vicente, E. (2019). *Introducción a la programación informática: Una excelente guía visual que explica, Paso a paso, Desde el Código Binario Hasta la Creación de Juegos*. Coordinación editorial de la Edición Española Elsa Vicente. Dorling Kindersley.